Video Presentation Guide & 5-Minute Script

Take-Home Assignment: User Management Dashboard with Custom Theming Engine

PART 1: Video Kaise Banayein (Setup & Tools)

1. Best Tool: Loom (Recommended & Free)

• Visit www.loom.com and use the free recorder (Screen + Camera bubble).
• Loom automatically generates a shareable web link when you stop recording.
• You can paste this link directly into the assignment submission form!
• Alternative: Use Windows Screen Recorder (Win + Alt + R) or Clipchamp and upload to Google Drive.

2. Screen Setup Before Starting

• Tab 1: Running frontend at http://localhost:5173
• Tab 2: VS Code open showing server/ and client/ folder structure
• Target duration: 5 to 6 minutes. Reviewers look for calm, clear architectural understanding.

PART 2: 5-Minute Word-by-Word Speaking Script

[0:00 - 0:45] INTRODUCTION & SCOPE (Screen: Browser Dashboard)

"Hi everyone! My name is Ujjwal, and this is my walkthrough for the Full Stack Developer Intern Take-Home assignment: a User Management Dashboard with a zero-dependency Custom Theming Engine.

When scoping this project for the 24-hour timeline, I focused on two core priorities: first, implementing a complete and reliable MERN CRUD workflow persisted end-to-end in MongoDB Atlas; and second, the main differentiator of this assignment: building a completely self-contained theming engine using native CSS variables and the browser FontFace API without using any third-party UI libraries.

I used modern JavaScript with Vite and React to eliminate compilation overhead and focus maximum effort on UI responsiveness, edge cases, and accessible contrast math."

Hindi Meaning: Apna intro dein, batayein ki 24h me solid MERN CRUD aur zero-dependency theming engine par focus kiya.

[0:45 - 1:30] ARCHITECTURE WALKTHROUGH (Screen: VS Code folders)

"Let me quickly walk you through the architecture in the code:

On the backend, we follow a clean Express MVC structure. The User.js model defines our Mongoose schema with email uniqueness and enum fields. userController.js handles our CRUD operations and a dedicated stats aggregation endpoint. validate.js catches invalid inputs before database calls, and errorHandler.js converts Mongoose errors into structured JSON responses.

On the frontend, React is cleanly componentized: all API calls live inside a dedicated api.js service consumed via a custom useUsers hook, and all theming state is orchestrated inside ThemeContext.jsx."

Hindi Meaning: VS Code me server MVC (models, controllers, routes, validation) aur frontend API service + context structure dikhayein.

[1:30 - 2:45] LIVE CRUD WORKFLOW DEMO (Screen: http://localhost:5173)

"Now let's demonstrate the application in the browser.

Here on the Dashboard, we have real-time metrics fetched from MongoDB: Total Users, Active accounts, Inactive accounts, New registrations this week, and role distribution bars.

Let's navigate to the User Directory to test our complete CRUD cycle:
1. Create: I'll click 'Add New User'. If I submit empty or invalid fields, client validation catches it immediately. Let's create 'Aryan Gupta' with role Developer and department Engineering. As soon as I submit, the user appears in our table, and the Dashboard total count updates live.
2. Read: Clicking the eye icon opens our read-only profile modal with full details.
3. Update: Let's edit Aryan's department to 'Product Architecture'. The change is saved directly to MongoDB Atlas.
4. Delete: Clicking delete opens our custom Confirmation Modal. Upon confirming, the user is permanently removed from MongoDB, a success toast notifies us, and our stats decrement live."

Hindi Meaning: Browser me Dashboard stats, Create User (with validation), View details, Edit department, aur Delete confirmation modal live run karein.

[2:45 - 4:00] CUSTOM THEMING ENGINE DEEP-DIVE (Screen: Theme Studio Drawer)

"Now for the primary differentiator of this assignment: our Custom Theming Engine.

This engine is built completely from scratch using native CSS custom properties on the root element composed with React Context:
1. Design Presets: We have 4 coordinated presets — Oceanic Blue, Neon Violet, Darcula Slate, and Emerald Forest. Notice how clicking a preset instantly transitions all surface cards, tables, text colors, and shadows across both pages without any lag.
2. Custom Button Color: Regardless of which preset is active, the user can pick any custom accent color. Notice our WCAG contrast calculation: if I pick a bright neon yellow or green, the button text automatically flips to dark slate. If I choose crimson or dark blue, it flips to white for optimal legibility.
3. Custom Font Upload: Using the browser's native FontFace API, users can upload a TTF, OTF, or WOFF font. The file is converted to Base64, registered dynamically with document.fonts.add(), and assigned to --font-family.
4. Persistence: Now let's refresh the page. Both the preset, custom button color, and uploaded font persist in localStorage and rehydrate immediately without any flash of unstyled content!"

Hindi Meaning: Theme studio drawer kholkar 4 presets, custom button color (text contrast), font upload, aur reload persistence demonstrate karein.

[4:00 - 5:00] BACKEND HIGHLIGHT & WRAP-UP

"One piece of the backend I'm particularly proud of is the aggregation endpoint /api/users/stats. Rather than making multiple database queries, it uses MongoDB aggregation to compute total counts, active ratios, and role breakdowns in a single query.

A technical challenge I overcame was font persistence. Because binary fonts cannot be saved as plain strings in localStorage, I built a Base64 encoding pipeline with file size limits, and rehydrated it synchronously on mount to eliminate layout shifting.

Given more time in a production environment, I would add server-side pagination with query limits for 100,000+ records, CSV export for user reports, and JWT authentication with role-based permissions.

Thank you so much for your time and for reviewing my project!"

Hindi Meaning: Backend aggregation & font persistence challenge explain karein, future roadmap batayein, aur thank you bolkar wrap up karein.